

Name : Moshitha Kolanjiyappan
Register Number : 192421205
ITA0513 – Computer Vision

CO2-AT1- Analytical Problem Solving

Histogram Processing Sandboxes

Q1. Interactive Histogram Equalization & Saturation

Tool Used : ImageJ

Task

Enhance the contrast of a low-contrast grayscale image using Global Histogram Equalization. Observe the histogram and Cumulative Distribution Function (CDF) to identify contrast improvement and any saturation in bright or dark regions.

Steps

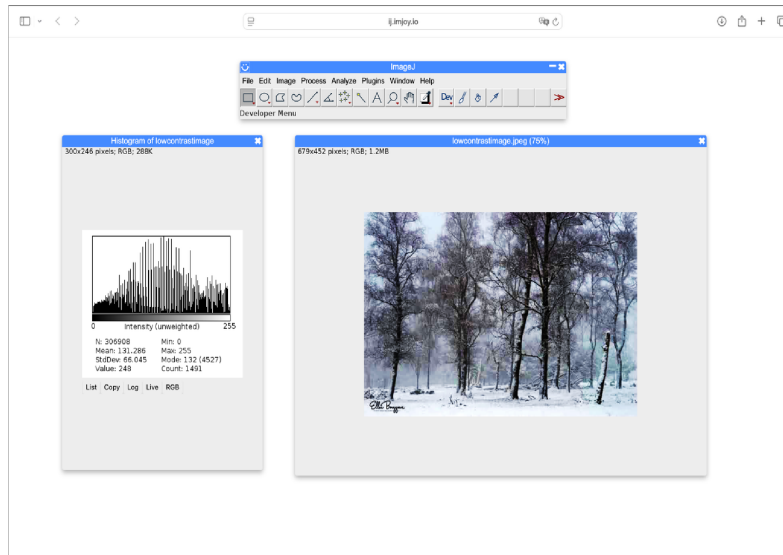
1. Open ImageJ.
2. Load the low-contrast grayscale image (File → Open).
3. Convert the image to grayscale (if required) using Image → Type → 8-bit.
4. View the original histogram using Analyze → Histogram.
5. Apply Process → Enhance Contrast and enable Equalize Histogram.
6. Observe the enhanced image and the updated histogram.
7. Compare the original and enhanced images and note any saturated regions.

Input Image



- Image Type: Low-contrast grayscale image
- Description: A snowy forest scene with trees. The image appears faded with poor contrast, making details difficult to distinguish.

Output Image



- After applying Global Histogram Equalization, the image contrast improved. The trees and snow became more distinct, and the intensity values were spread over the full grayscale range. A small amount of saturation ($\approx 0.45\%$) occurred in the brightest snow and darkest tree regions.



Spatial Domain Filtering Sandboxes

Q3. Convolution Kernel Customization

Tool Used: Setosa.io Image Kernels Interactive Visualizer

Task

Customize the default Blur kernel to create a horizontal motion blur filter while preserving the overall brightness of the image. Observe how the convolution kernel affects the edge intensity and image appearance.

Steps

1. Open the Setosa.io Image Kernels Interactive Visualizer.
2. Load the grayscale flower image.
3. Select the Blur preset.
4. Modify the 3×3 kernel values to create a horizontal motion blur

Input Image



- Image Type: Grayscale flower image.
- Description: A grayscale image of a flower with clear edges and smooth background, used to demonstrate the effect of convolution kernels.

Output Image

setosa.io

Before, for each one block of pixels in the image on the left, we multiply each pixel by the corresponding entry in the kernel and then take the sum. That sum becomes a new pixel in the image on the right. Hover over a pixel on either image to see how its value is computed.

input image

output image

kernel: blur

One subtlety of this process is what to do along the edges of the image. For example, the top left corner of the input image only has three neighbors. One way to fix this is to extend the edge values out by one in the original image while keeping our new image the same size. In this demo, we've instead ignored those values by making them black.

Here's a playground where you can select different kernel matrices and see how they effect the original image or build your own kernel. You can also upload your own image or use live video if your browser supports it.

Choose File flower-kernel.jpg

0.0625	0.125	0.0625
0.125	0.25	0.125
0.0625	0.125	0.0625

blur



The **blur** kernel de-emphasizes differences in adjacent pixel values.

For more, have a look at Gimp's excellent documentation on using [image kernel's](#). You can also apply your own custom filters in

- After applying the customized horizontal motion blur kernel, the image became smoothly blurred in the horizontal direction while maintaining its overall brightness. The vertical edges of the flower appeared softer with gradual intensity transitions, whereas the horizontal structures remained relatively clear.



Frequency Domain Visualizers

Q6. 2D Fast Fourier Transform (FFT) Phase Filtering

Tool Used: Paul Falstad's 2D Fourier Transform (FFT) Simulator Input

Task

Apply 2D Fast Fourier Transform (FFT) to a grayscale image and remove the high-frequency components by masking the outer regions of the Fourier spectrum. Observe how the reconstructed image changes after inverse transformation and compare it with a Gaussian low-pass blur effect.

Steps

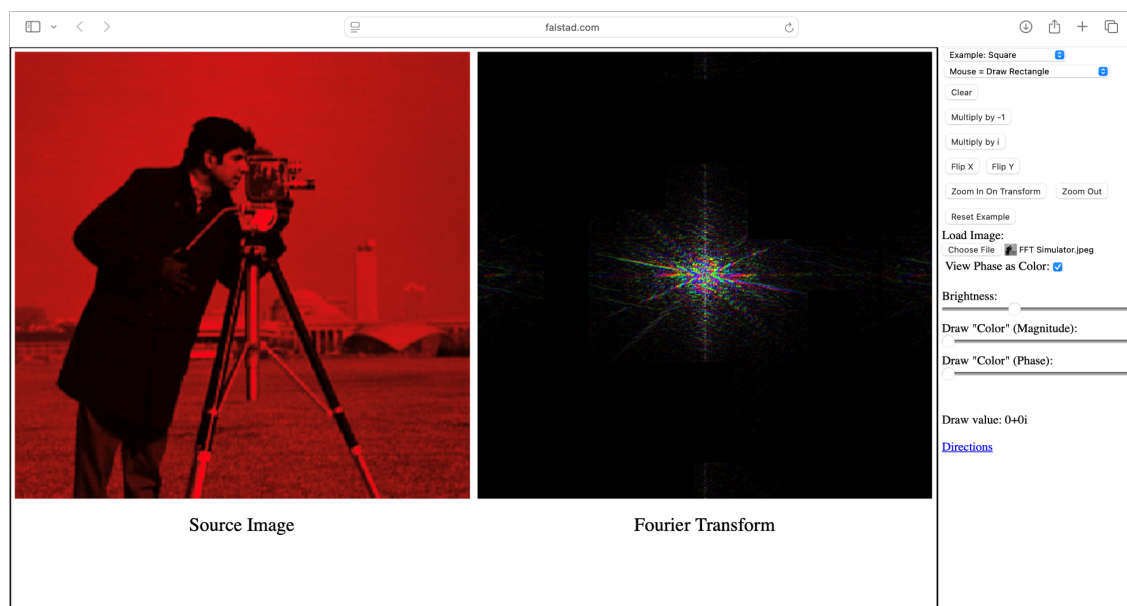
1. Open Paul Falstad's 2D Fourier Transform (FFT) Simulator.
2. Load the grayscale Cameraman image.
3. View the Fourier Transform (frequency spectrum) of the image.
4. Keep the bright center (low-frequency components) unchanged.
5. Mask (erase) the high-frequency components located at the outer regions of the Fourier spectrum.
6. Allow the simulator to reconstruct the image automatically.
7. Compare the original and reconstructed images and observe the loss of fine details and edges.

Input Image



- Image Type: Grayscale Cameraman image.
- Description: A standard grayscale image containing fine details and sharp edges.

Output Image



- After masking the high-frequency components in the Fourier spectrum, the reconstructed image became smoother and less detailed. Most of the fine textures and sharp edges were removed, while the overall structure of the image remained visible. This behavior is similar to applying a Gaussian low-pass blur filter.

Q8. Python OpenCV Edge Enhancement Sandbox

Tool Used : Visual Studio Code (VS Code) with Python, OpenCV, NumPy, and Matplotlib

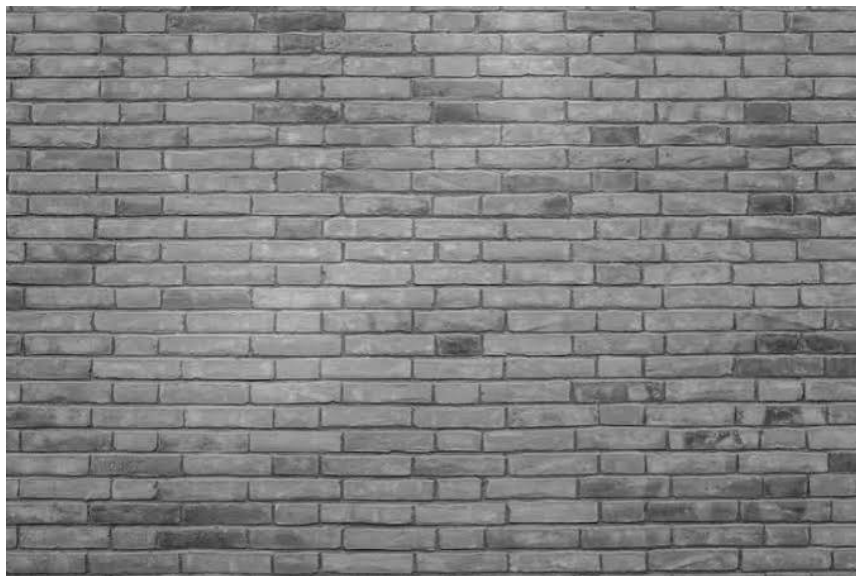
Task

Apply Sobel X, Sobel Y, and Laplacian edge detection operators on a grayscale bricks image. Compute the Gradient Magnitude by combining the Sobel derivatives and compare the results obtained using 3×3 and 5×5 Sobel kernels to analyze the effect of kernel size on edge thickness.

Steps

1. Open Visual Studio Code.
2. Place the grayscale bricks image in the project folder.
3. Import the required libraries: OpenCV, NumPy, and Matplotlib.
4. Load the image in grayscale using `cv2.imread()`.
5. Apply the Sobel X and Sobel Y operators with a 3×3 kernel.
6. Compute the Gradient Magnitude using the Sobel X and Sobel Y outputs.
7. Apply the Laplacian operator to detect edges in all directions.
8. Change the Sobel kernel size from 3×3 to 5×5 and compute the new Gradient Magnitude.
9. Display the Original Image, Sobel X, Sobel Y, Laplacian, Gradient Magnitude (3×3), and Gradient Magnitude (5×5) for comparison.

Input Image



- Gradient Magnitude (3×3): Combines Sobel X and Sobel Y to display the overall edge strength with thin and sharp edges.
- Gradient Magnitude (5×5): Produces thicker and smoother edges with reduced noise compared to the 3×3 kernel.